

Sistem ve Network Atölyesi

High Availability And Scalability



Ajanda

- High Availability
 - High Availability and Fault Tolerance
 - Elastic Load Balancer (ELB)
 - Application Load Balancer(ALB)
- Scalability
 - EC2 Auto Scaling Temelleri
 - Ölçekleme Politikaları
 - İzleme ve Uyarı
- Hands-on
 - Multi-AZ Auto Scaling Web App

High Availability

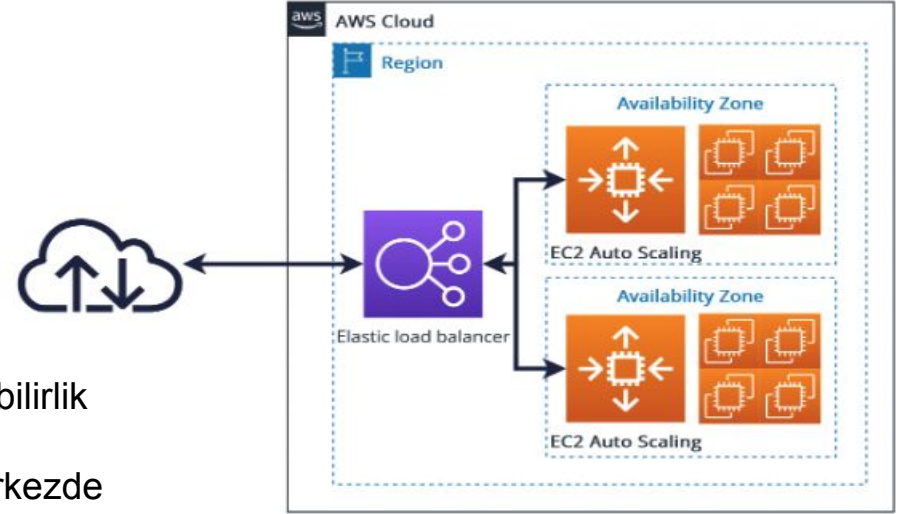


High Availability

High Availability Nedir?

Bir nesnenin veya kaynağın yedekli kopyalarının oluşturulması ve biri başarısız olduğunda diğerinin devralmasıdır.

- AWS'de yüksek erişilebilirlik Multi-AZ (Çoklu Erişilebilirlik Alanı) dağıtımı ile sağlanır
- Veri merkezleri coğrafi olarak dağıtıldığı için bir merkezde yaşanan sorun diğerlerini etkilemez



Yüksek Erişilebilirlik ve Hata Toleransı – Tek Bölge (Single Region)

Tanımlar

- **Yüksek Erişilebilirlik (HA):** Arızalarda **en az kesinti** ile çalışmaya devam etme.
- **Hata Toleransı (Fault Tolerance):** Bir bileşen bozulsun bile **hiç kesinti olmadan** çalışmaya devam etme.

Nasıl? (Multi-AZ)

- **Availability Zone (AZ):** Aynı bölgede, birbirinden izole veri merkezleri.
- **Destekleyen servisler:** EC2, RDS, ELB/ALB, **Auto Scaling Group (ASG)** çok-AZ kurulumları.

Örnek Mimari

- **Uygulama:** En az **2 AZ**'de EC2, önünde **ALB**.
- **Veritabanı:** **RDS Multi-AZ** (otomatik failover).

Tek Bölge (Single Region) HA ve Hata Toleransı

Hata Toleransı (Fault Tolerance)

Bileşenler arızalandığında **uygulamanın kesintisiz çalışmaya devam etmesini** sağlar.

Nasıl Sağlanır:

- Otomatik replikasyon ve failover (yük devretme) mekanizmaları ile

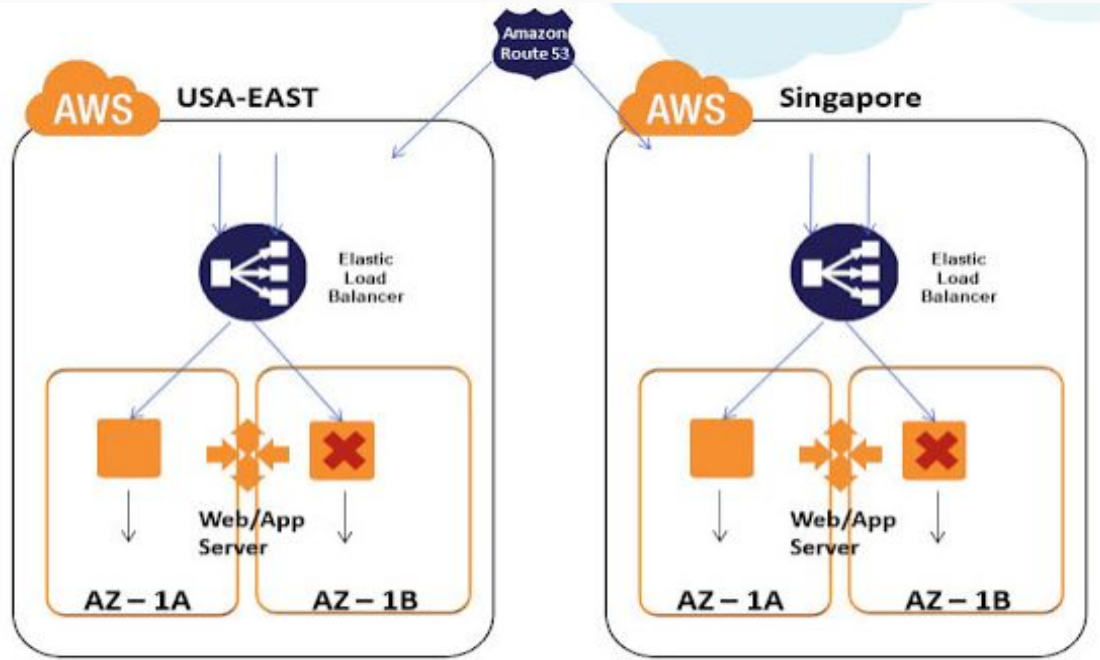
Örnekler:

- **Amazon S3:** Verileri otomatik olarak birden fazla AZ'ye kopyalar
- **Amazon DynamoDB ve SQS:** Veri replikasyonu ile yerleşik hata toleransı sunar

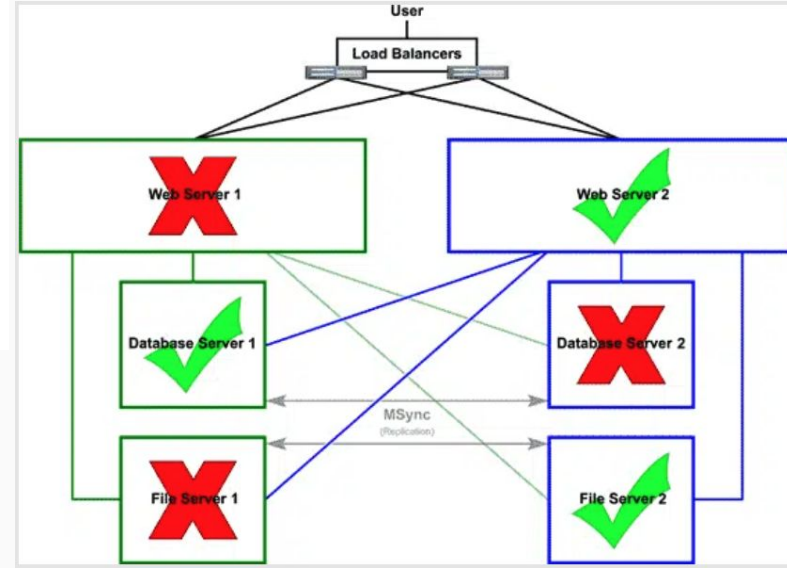
✓ En İyi Uygulamalar:

- Trafiği AZ'ler arasında dağıtmak için **Elastic Load Balancer** kullanın
- Talep artışları veya arızalarda **Auto Scaling** kullanın
- Kolay failover için **stateless servisler** tasarlayın
- Dayanıklı veritabanı katmanları için **multi-AZ RDS veya Aurora** kullanın
- **Amazon CloudWatch** ile sağlık durumunu izleyin, alarm ve kurtarma aksiyonları yapılandırın

High Availability and Fault Tolerance



Yüksek erişilebilirlik sistemi, çeşitli bileşenlerin arızalanması durumunda bile çalışmaya devam edebilir



Neden Multi-AZ yetmez; Multi-Region gerekir?

!!AWS is Down & let's understand How? 😞

AWS Outage (US-EAST-1): Explained

Today, a major AWS outage affected many apps and services around the world. Here's a quick and easy explanation of what went wrong:

- 1 It all started with a DNS problem, that means AWS services couldn't find or connect to each other.
- 2 This caused DynamoDB to stop working. Many other AWS tools and customer applications depend on it, so they also started failing.
- 3 As a result, EC2 instances (virtual servers) and Lambda functions (serverless code) couldn't start or run properly.
- 4 AWS fixed the DNS issue, but some services are still recovering and syncing up.

This outage shows how even a small issue in one part of the cloud can affect many others, and how much of the internet depends on these systems. 🌩️

Follow live updates: <https://lnkd.in/eDbfJTvv>

#aws



AWS engineers right now: doing Diwali Puja to the

AWS Health Dashboard

Updated less than 1 minute ago

View your account health

Get a personalized view of events that affect your AWS account or organization.

Open your account health

Service health

View the current and historical status of all AWS services.

Open and recent issues (1)

Service history

▼ Operational issue - Multiple services (N. Virginia)

Service: Multiple services

Severity: Degraded

RSS

Increased Error Rates and Latencies

Oct 20 11:22 AM PDT Our mitigations to resolve launch failures for new EC2 instances continue to progress and we are seeing increased launches of new EC2 instances and decreasing networking connectivity issues in the US-EAST-1 Region. We are also experiencing significant improvements to Lambda invocation errors, especially when creating new execution environments (including for Lambda@Edge invocations). We will provide an update by 12:00 PM PDT.

Oct 20 10:38 AM PDT Our mitigations to resolve launch failures for new EC2 instances are progressing and the internal subsystems of EC2 are now showing early signs of recovering in a few Availability Zones (AZs) in the US-EAST-1 Region. We are applying mitigations to the remaining AZs at which point we expect launch errors and network connectivity issues to subside. We will provide an update by 11:30 AM PDT.

Oct 20 10:03 AM PDT We continue to apply mitigation steps for network load balancer health and recovering connectivity for most AWS services. Lambda is experiencing function invocation errors because an internal subsystem was impacted by the network load balancer health checks. We are taking steps to recover this internal Lambda system. For EC2 launch instance failures, we are in the process of validating a fix and will deploy to the first AZ as soon as we have confidence we can



AWS OUTAGE (20 OCTOBER 2025)

Söz konusu kesinti, 1000'den fazla şirketi etkiledi.

Amazon & Amazon AWS

Pinterest

HMRC

Snapchat

Vodafone

Roblox

PlayStation

Fortnite

Pokémon Go.

Duolingo

Coinbase

Canva

Downdetector

<https://edition.cnn.com/business/live-news/amazon-tech-outage-10-20-25-intl>

Çok Bölge (Multi-Region) HA & Fault Tolerance

Bölgeler Arası Yüksek Erişilebilirlik

Çoklu bölge HA, **tüm bir AWS bölgesi çökse bile** uygulamanızın erişilebilir ve yanıt verebilir kalmasını sağlar.

Mimari Yaklaşım:

- İş yüklerini iki veya daha fazla AWS bölgesine dağıtın (örn: **us-east-1** ve **eu-west-1**)
- **Amazon Route 53** ile sağlık kontrolleri, gecikme tabanlı yönlendirme (latency-based routing) veya failover routing kullanın
- **Active-Active** (yük paylaşımı) veya **Active-Passive** (birincil/yedek) modelleri uygulayın



Örnek Senaryo:

- Web uygulamalarını iki farklı bölgede dağıtın
- Route 53 failover routing kullanın: Bölge A çökerse, Route 53 otomatik olarak Bölge B'ye yönlendirir

Çok Bölge (Multi-Region) HA & Fault Tolerance

Bölgeler Arası Hata Toleransı

Hata toleransı, **tüm bir bölge kullanılamaz hale gelse bile** uygulamanın sorunsuz çalışmaya devam etmesini sağlar.

Replikasyon Stratejileri:

- Veri ve servisleri bölgeler arasında kopyalayın
- **Amazon S3 Cross-Region Replication (CRR)**, Aurora Global Databases, DynamoDB Global Tables kullanın
- Global içerik dağıtımı için **Amazon CloudFront** kullanın
- Düşük RPO (Recovery Point Objective) için asenkron veri replikasyonu sağlayın



Örnekler:

- **DynamoDB Global Tables:** Bölgeler arası tam replikasyonlu, multi-master veritabanları
- **Aurora Global Databases:** Saniyenin altında gecikme ile replikasyon

High Availability with Route 53 DNS Failover

your web application spans over multiple aws regions.

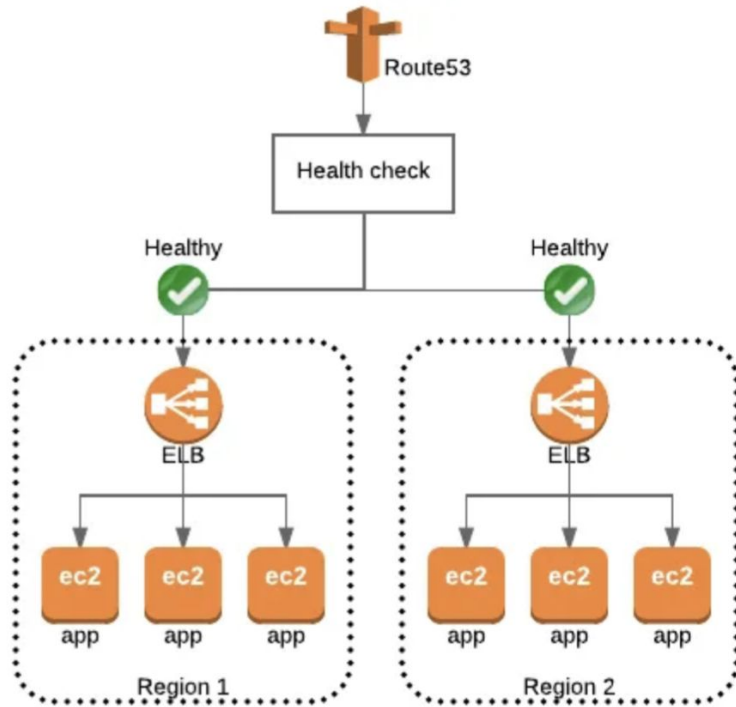


Figure 1 - Both regions operating normally

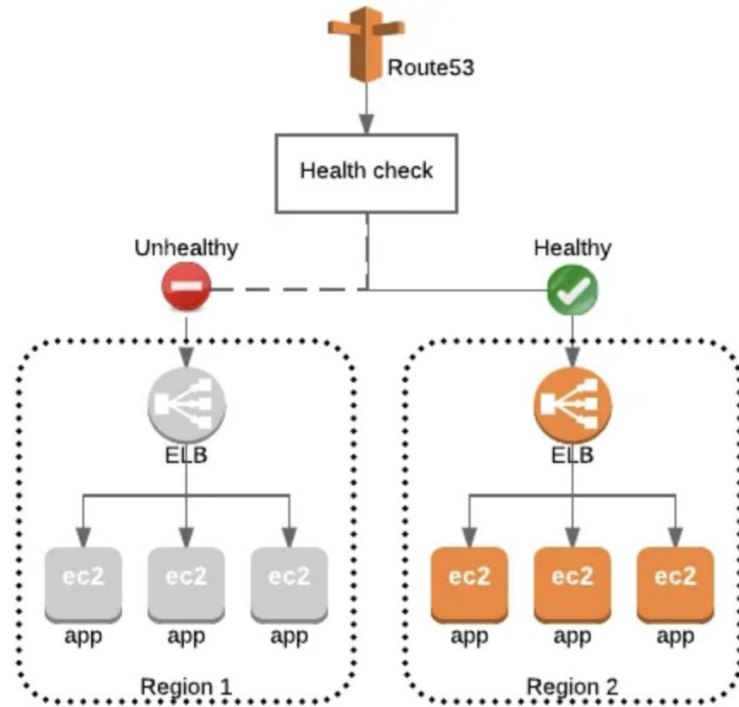
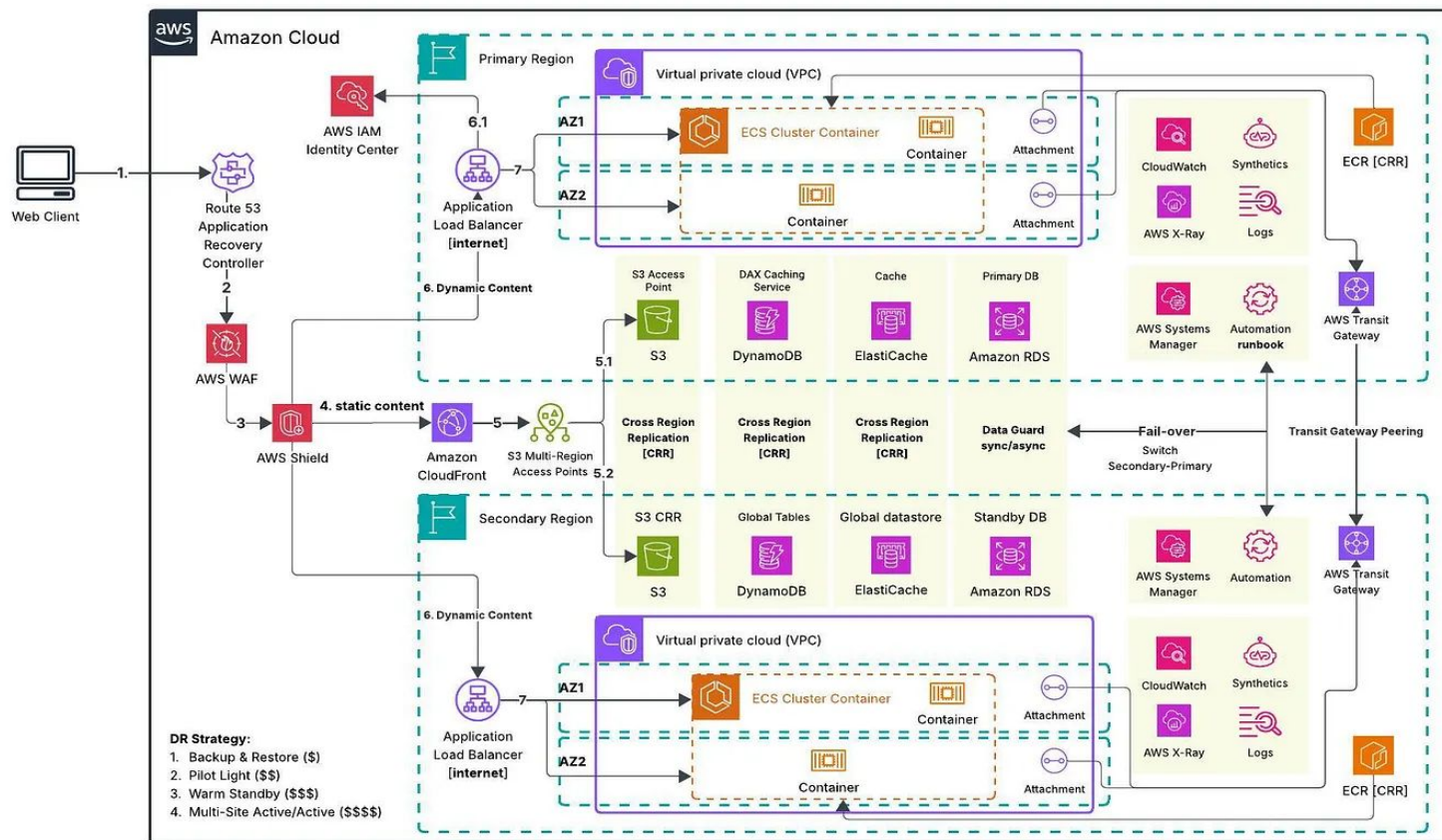


Figure 2 - region 1 experiencing issues

I'll be using the latency based failover for this setup. With latency-based routing, Amazon Route 53 can direct your users to the lowest-latency AWS

Multi Region High Availability and Fault Tolerance Architecture



Elastic Load Balancing in AWS

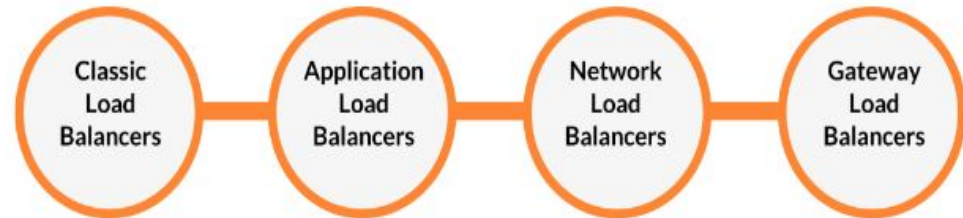
AWS Elastic Load Balancing (ELB)

Elastic Load Balancing, AWS tarafından sunulan yönetilen bir yük dengeleme hizmetidir. Gelen trafiği EC2 örnekleri, konteynerler ve bir veya daha fazla Erişilebilirlik Alanı'ndaki IP adresleri gibi farklı kaynaklar arasında dağıtır.

AWS Load Balancer Types

4 Types of Load Balancers in AWS

- Classic Load Balancer
- Application Load Balancer
- Network Load Balancer
- Gateway Load Balancer



AWS Load Balancer Temel Kavramları

Target Group Nedir?

- EC2 instance'ları, Lambda fonksiyonları, EKS cluster'ları, IP adresleri gibi hedeflerin mantıksal bir gruptandır
- Load balancer trafiği bu gruplara dağıtır

Listeners (Dinleyiciler)

Listener, load balancer'ın **hangi port ve protokolde** istekleri dinleyeceğini belirleyen yapılandırmadır.

- **Frontend (Kullanıcı tarafı):** Load balancer'ın hangi port ve protokolde (HTTP:80, HTTPS:443) istekleri kabul edeceğini belirler
- **Backend (Sunucu tarafı):** Gelen isteklerin hangi hedef gruplara, hangi protokol ve port üzerinden iletileceğini tanımlar

Kullanıcı → HTTPS:443 (Listener) → Load Balancer → HTTP:80 → Target Group → EC2 Instance'lar

AWS Load Balancer Temel Kavramları

Health Checks (Sağlık Kontrolleri)

Health check, load balancer'ın backend hedeflerinin durumunu kontrol ederek ****sadece sağlıklı hedeflere trafik göndermesini**** sağlayan mekanizmadır.

Health Check Nasıl Çalışır?

- Load balancer, belirli aralıklarla hedeflere istek gönderir
- Yanıt alırsa hedef "healthy" (sağlıklı), alamazsa "unhealthy" (sağlıksız) olarak işaretlenir
- Sağlıksız hedeflere trafik gönderilmez

Desteklenen Protokoller

HTTP, HTTPS ve TCP protokollerini destekler -

Örnek: `/health` endpoint'ine HTTP GET isteği gönderip 200 OK yanıtı bekler

Örnek Senaryo:

Health Check: HTTP:80/health

Interval: 30s

Timeout: 5s

Healthy Threshold: 3

Unhealthy Threshold: 2

Network Load Balancer

Aşırı Yüksek Trafik İçin Tasarlandı

Network Load Balancer, **yüksek performanslı uygulamalar** için ideal bir seçimidir:

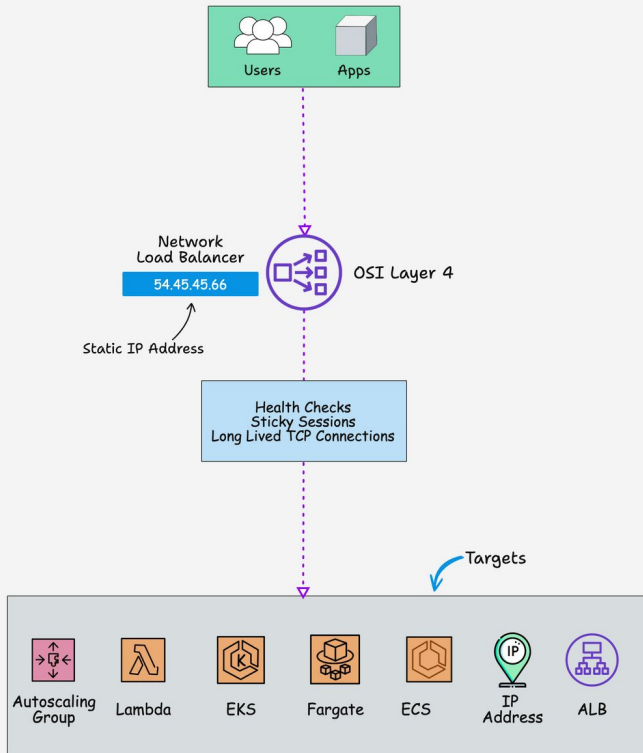
- **Değişken trafik desenlerini** yönetmek üzere tasarlanmıştır
- **Saniyede milyonlarca istek** işleyebilir
- Ultra düşük gecikme süresi (latency) sağlar

Ne Zaman Kullanılır?

- Oyun sunucuları (gaming)
- Gerçek zamanlı video streaming
- Yüksek frekanslı finansal işlemler -
- IoT cihazlarından gelen yüksek hacimli veri

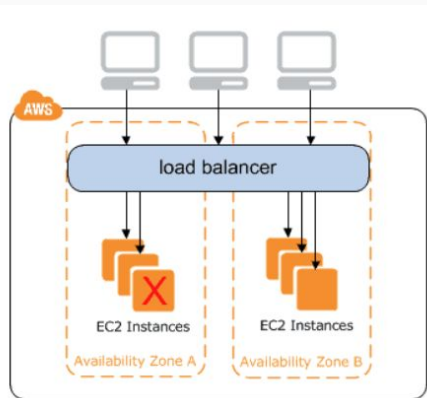
NLB Protokol ve Katman Özellikleri

- TCP ve UDP Protokollerini Destekler
- Statik IP Adresi Desteği
- Security Group Desteği YOK
- OSI Katman 4'te Çalışır
- Tek EC2 Instance'da Çoklu Uygulama Desteği



Classic Load Balancer

CLB **legacy**'dir; yeni mimarilerde **ALB/NLB** kullanın.



Neden Legacy?

- AWS, yeni özellikler için CLB'yi güncellemiyor
- ALB ve NLB daha fazla özellik ve esneklik sunar
- Mevcut CLB'ler çalışmaya devam eder, ancak yeni projeler için tercih edilmemelidir

CLB'den ALB/NLB'ye Geçiş

Neden Geçiş Yapmalısınız?

Özellik	CLB	ALB	NLB
OSI Layer	Layer 4 & 7 (karma)	Layer 7	Layer 4
Path-based routing	✗	✓	✗
Host-based routing	✗	✓	✗
WebSocket	✗	✓	✓
HTTP/2	✗	✓	✗
Static IP	✗	✗	✓
Container support	Sınırlı	✓	✓
Lambda target	✗	✓	✓

Geçiş Önerisi:

Mevcut CLB → ALB/NLB'ye migrate edin

- ALB: HTTP/HTTPS uygulamaları için
- NLB: TCP/UDP ve yüksek performans için

Gateway Load Balancer

🛡️ GWLB - Güvenlik ve Ağ Cihazları İçin Özel Load Balancer

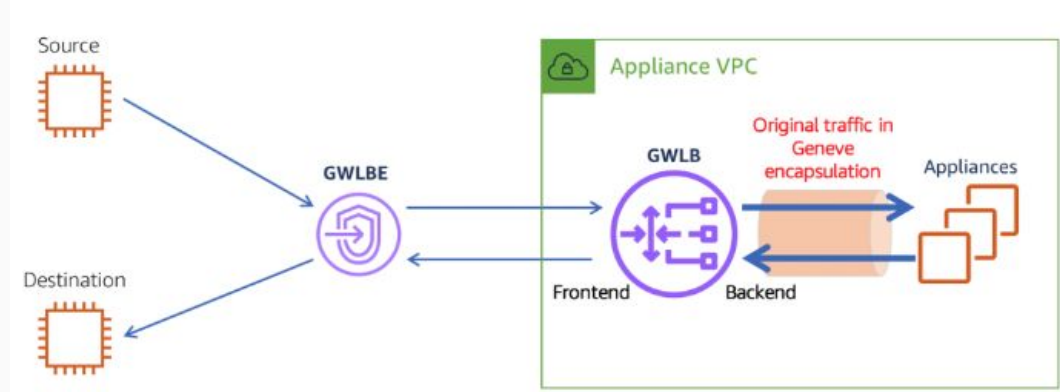
Gerçek Dünya Kullanım Senaryoları

Gateway Load Balancer, özellikle **güvenlik ve ağ yönetimi** için tasarlanmıştır:

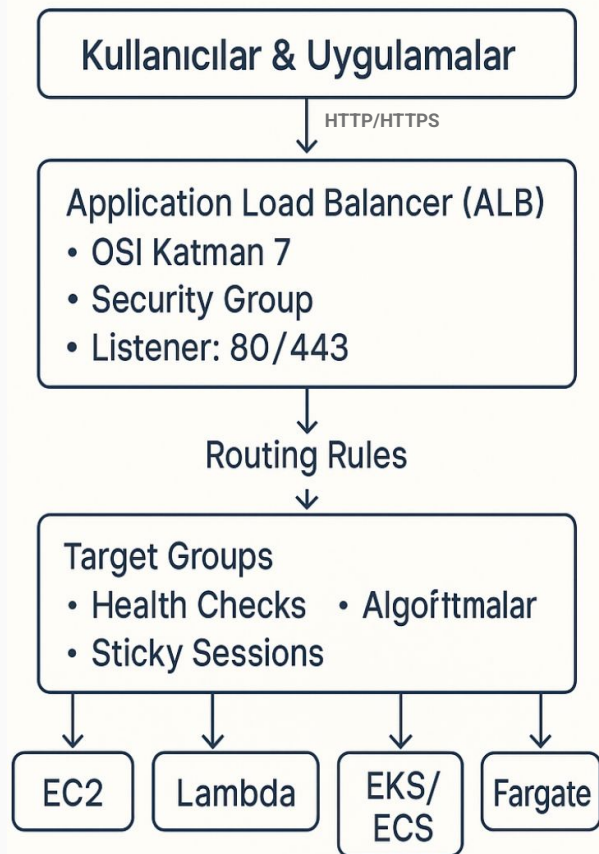
- **Firewall Yönetimi:** Gelişmiş güvenlik duvarı dağıtımı
- **Intrusion Detection Systems (IDS):** Saldırı tespit sistemleri
- **Intrusion Prevention Systems (IPS):** Saldırı önleme sistemleri
- **Deep Packet Inspection (DPI):** Derin paket inceleme
- **Network Monitoring:** Ağ trafiği izleme ve analiz

ALB/NLB: Uygulama trafiğini dağıtır

GWLB: Trafiği güvenlik cihazlarından geçirir ve geri döndürür



Application Load Balancer



ALB, OSI Katman 7'de Çalışır (Uygulama Katmanı)

- HTTP/HTTPS isteklerinin **içeriğine bakarak** akıllı yönlendirme yapar
- URL path'i, hostname, HTTP header'ları gibi bilgileri analiz edebilir

Gelişmiş Yönlendirme (Routing) Kuralları

ALB üç temel yönlendirme türünü destekler:

- **Path-based routing:** `/api/*` → API sunucularına, `/images/*` → CDN'e
- **Host-based routing:** `api.example.com` → API target group, `web.example.com` → Web target group
- **Query string routing:** `?version=v2` parametresine göre yönlendirme

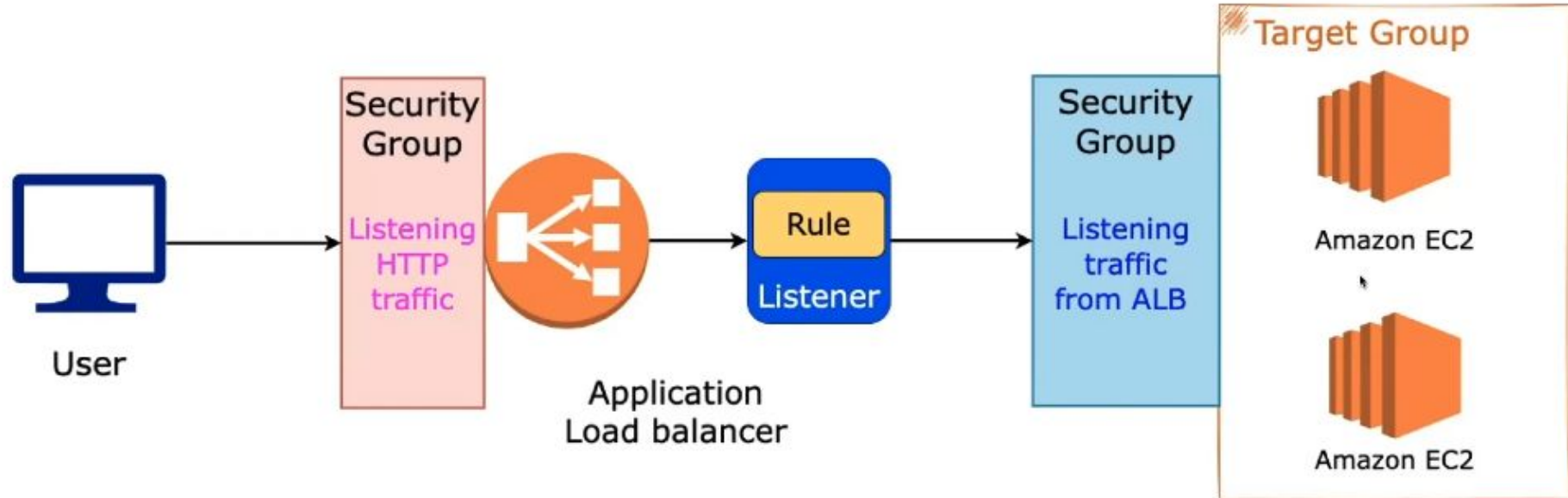
Security Group Entegrasyonu

ALB'ye **Security Group** ekleyerek gelen trafiği kısıtlayabilirsiniz -

Stickiness (Yapışkan Oturum) Özelliği

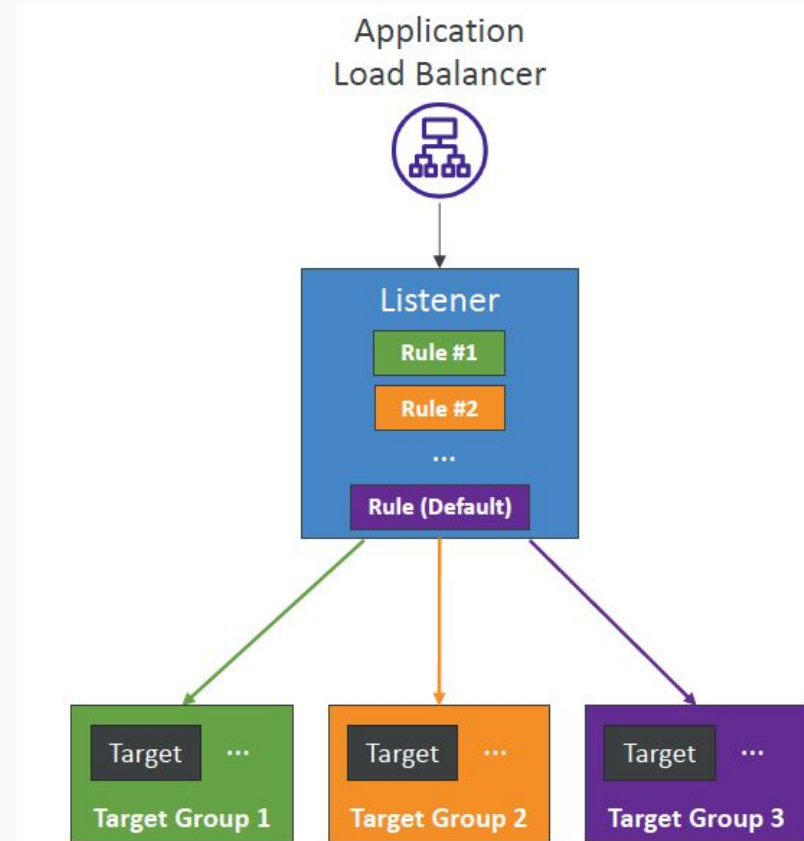
Belirli bir kullanıcıdan gelen tüm isteklerin **aynı backend sunucusuna** yönlendirilmesini sağlar

AWS Application Load Balancer



ALB- Listener Rules

- Processed in order (with Default Rule)
- Supported Actions (forward, redirect, fixed-response)
- Rule Conditions:
 - host-header
 - http-request-method
 - path-pattern
 - source-ip
 - http-header
 - query-string



ELB

Feature	Application Load Balancer	Network Load Balancer	Classic Load Balancer
Protocols	HTTP, HTTPS	TCP	TCP, SSL/TLS, HTTP, HTTPS
Platforms	VPC	VPC	EC2-Classic, VPC
Health checks	✓	✓	✓
CloudWatch metrics	✓	✓	✓
Logging	✓	✓	✓
Path-Based Routing	✓		
Host-Based Routing	✓		
Native HTTP/2	✓		
			✓
SSL offloading	✓		✓
Static IP		✓	
Elastic IP address		✓	
Slow start	✓		

Performans ve Kapasite Karşılaştırması

Metrik	ALB	NLB	GWLB	CLB
Request Per Second	~100,000	Milyonlarca	Yüksek	~10,000
Latency	10-100 ms	<100 μ s	Düşük	50-200 ms
Connection Handling	HTTP/1.1, HTTP/2	Long-lived TCP	Stateful inspection	HTTP/1.0, HTTP/1.1
Scalability	Otomatik	Otomatik	Otomatik	Otomatik
Cost Efficiency	Orta	Yüksek (throughput başı)	Yüksek	Düşük

Özet Tavsiyeler

Durum	Öneri
Yeni web uygulaması	ALB (Path/Host routing)
Microservice architecture	ALB (Container support)
Gaming/VoIP	NLB (Ultra-low latency)
IoT	NLB (Milyonlarca connection)
Merkezi firewall	GWLB (Security appliances)
Legacy uygulama	CLB → ALB/NLB'ye migrate et
Statik IP + HTTP	NLB + ALB kombinasyonu
Compliance + Inspection	GWLB + ALB kombinasyonu

Scalability(Auto Scaling)

Auto Scaling

AutoScaling Group(ASG)

Trafiğe ve sağlık durumuna göre **EC2 instance'larını** **otomatik ekleyen/çıkarın** mekanizma.

Bileşenler:

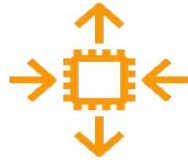
- **Launch Template:** AMI, instance tipi, EBS, SG, IAM role, user-data.
- **Auto Scaling Group:** Min/Desired/Max kapasite, VPC & Subnet'ler, çok-AZ yerleşim, ALB/Target Group entegrasyonu, health check.
- **Scaling Policies:** Target Tracking, Step, Scheduled, (opsiyonel) Predictive

Faydalar:

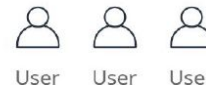
- **Esnek kapasite:** scale-out, scale-in.
- **Yüksek erişilebilirlik:** Çok-AZ dağılımı
- **Maliyet optimizasyonu:** Doğru zamanda doğru boyut.
- **Kesintisiz güncelleme:**

we can configure automatic scaling for all of the scalable resources :

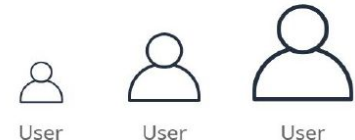
- Amazon EC2
- Amazon EC2 Spot Fleets
- Amazon ECS
- Amazon DynamoDB
- Amazon Aurora



It scales across multiple Availability Zones within the same AWS region.

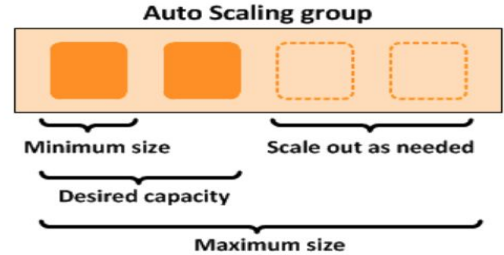
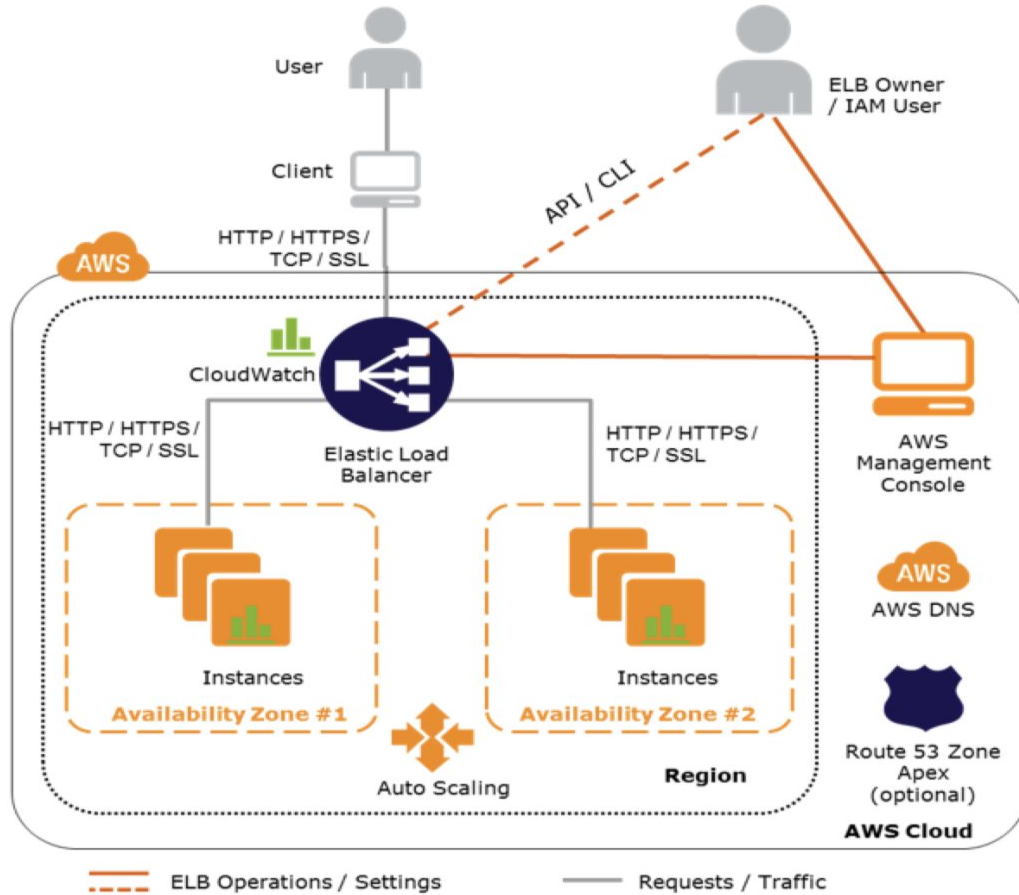


Horizontal Scaling



Vertical Scaling

EC2 Auto Scaling



Auto Scaling-Configuration

AWS Auto Scaling Components

Launch Template

- AMI (AWS veya Custom)
- Instance Type
- EBS Volumes
- Security Groups
- **IAM Role** (instance profile)
- **Key Pair**
- **User Data** (bootstrap)
- **Metadata/IMDSv2**, Placement, ENI, vb.

Auto Scaling Group

- Launch Template (+version)
- VPC & Subnets (çok-AZ)
- Min/Desired/Max
- Load Balancer / Target Group, Health check (ALB)
- Scaling Policies: **Target Tracking**, **Step**, **Scheduled**, **Predictive**
- Cooldown, Termination policy, Capacity Rebalance
- **Warm Pool** (opsiyonel)
- Monitoring & Alarms (CloudWatch)

AWS Auto Scaling Components

Scaling Policies

- **Target Tracking:** Tek hedefi sabit tut (örn. CPU %50, P95 < 350 ms).
- **Step Scaling:** Eşiklere göre adımlı artış/azalış (örn. RPS > 200 $\Rightarrow$ +4).
- **Scheduled Scaling:** Saat bazlı ön-ısıtma/azaltma (kampanya, mesai).
- **Predictive Scaling:** Geçmiş trafiğe göre önceden kapasite ayarı.



Target
Tracking



Scheduled
Scaling

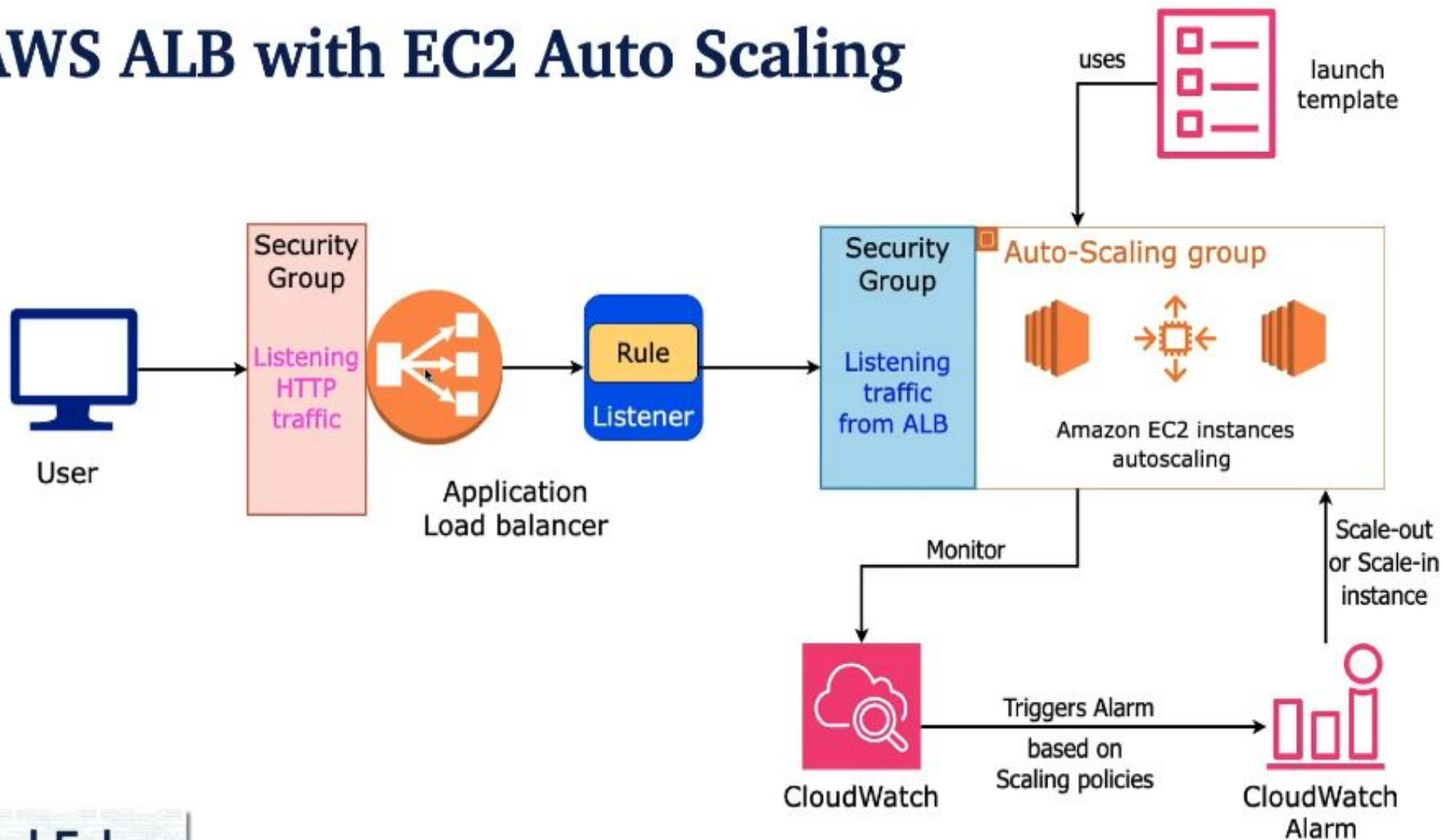


Step
Scaling



Predictive
ML

AWS ALB with EC2 Auto Scaling



CloudWatch & SNS

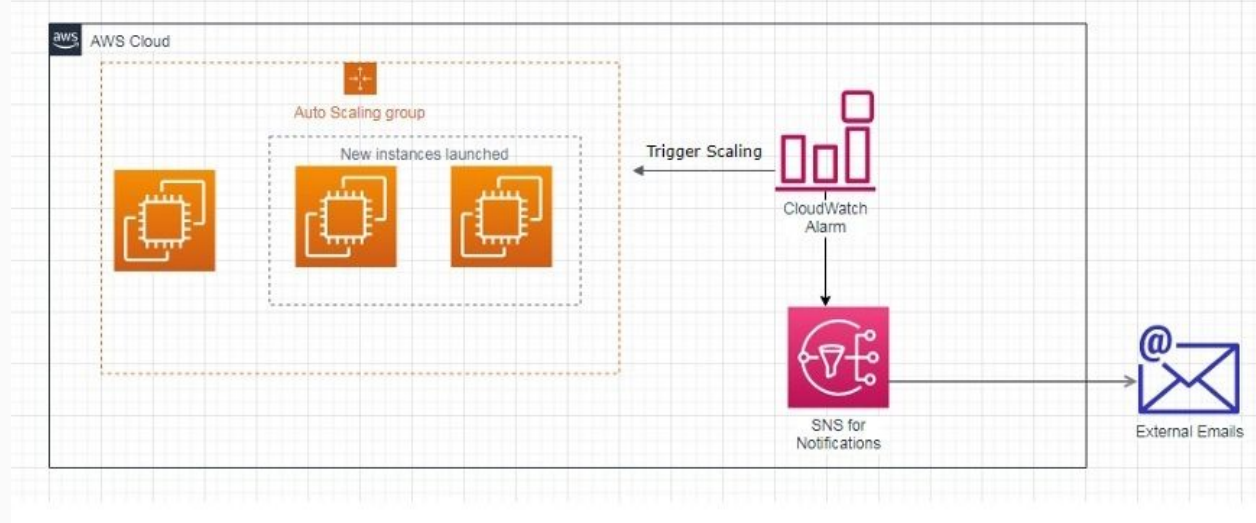
CloudWatch: AWS monitoring servisi

- CPU, bellek, disk, ağ kullanımı ölçümü
- Custom metric eklenebilir

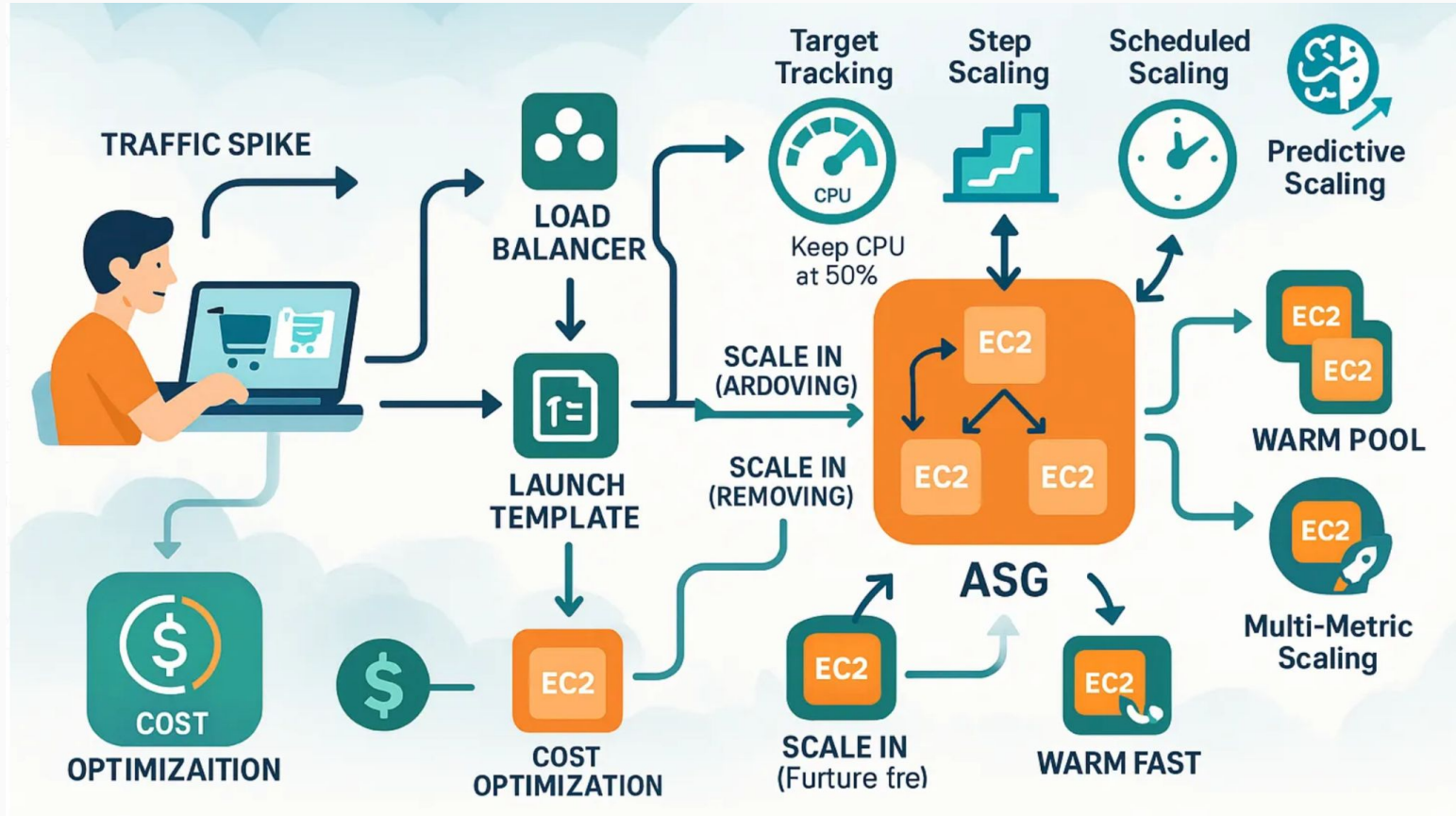
SNS (Simple Notification Service):

- CloudWatch alarm → SNS topic → e-posta/SMS/Slack bildirimi

Örnek: CPU %80 → ASG scale-out + admin'e e-posta



How Auto Scaling Works



Multi-AZ Auto Scaling Web App



Multi-AZ Auto Scaling Web App

- **VPC hazırla**
- **Bastion Sunucu kur**
- **Launch Template oluştur (Amazon Linux + Apache)**
- **target group kur**
- **ALB kur**
- **ASG kur (Min:1, Max:3, Desired:1)**
- **ASG'yi target group'a ve ALB'ye bağla**
- **CloudWatch CPU alarm kur (örn. %70 scale-out, %30 scale-in)**
- **SNS notification ile e-posta bildirimi ekle**
- **Load test → yeni instance otomatik ekleniyor mu kontrol et**

Multi-AZ Auto Scaling Web App

- **VPC hazırla**
- **Bastion Sunucu kur**
- **Launch Template oluştur (Amazon Linux + Apache)**
- **target group kur**
- **ALB ekle ve ASG'yi target group'a bağla**
- **ASG kur (Min:1, Max:3, Desired:1)**
- **CloudWatch CPU alarm kur (örn. %70 scale-out, %30 scale-in)**
- **SNS notification ile e-posta bildirimi ekle**
- **Load test → yeni instance otomatik ekleniyor mu kontrol et**

TERRAFORM

Terraform Nedir?



Infrastructure as Code (IaC) nedir?

Terraform = HashiCorp tarafından geliştirilen açık kaynaklı IaC aracı

Ne işe yarar?

- **Cloud altyapısını (AWS, Azure, GCP, vs.) kodla tanımlama**
- **Otomatik, güvenli ve tekrar edilebilir kaynak oluşturma**

Neden Terraform?

! **Deklaratif yapı** (Ne istendiğini söylersin, nasıl yapılacağını değil)

☁ **Cloud Provider bağımsızlığı** (AWS, GCP, Azure)

↺↻ **Tekrar edilebilirlik ve sürüm kontrolü**

🔒 **Güvenli ve denetlenebilir yapı**

Terraform Temel Yapısı



`.tf` dosyaları (Terraform configuration files)

`provider` → AWS, Azure, vs.

`resource` → Oluşturulacak altyapı nesneleri (EC2, S3, RDS..)

`variable`, `output`, `module` gibi yapı taşları

Terraform Workflow - 4 Temel Adım

1. `terraform init` – Başlatma (provider'ları indir)
2. `terraform plan` – Değişiklikleri göster
3. `terraform apply` – Değişiklikleri uygula
4. `terraform destroy` – Altyapıyı sil

Basit Bir Örnek - AWS EC2

```
provider "aws" {  
    region = "us-east-1"  
}  
  
resource "aws_instance" "example" {  
    ami  
        =  
"ami-0c55b159cbfafe1f0"  
    instance_type = "t2.micro"  
}
```

Terraform State Dosyası

`terraform.tfstate` – Terraform'un altyapıyı izlediği dosya

Lokal veya Remote (S3 + DynamoDB gibi)

`terraform refresh, terraform state list`

Terraform ile Modülerlik



`modules` nedir?

Örnek: VPC, EC2 gibi kaynaklar modül haline getirilerek tekrar kullanılabilir

Büyük altyapılarda düzen sağlar

Terraform Backend - State Yönetimi



Neden önemli?

- Takım çalışmasında state çakışmalarını önler

Remote backend: AWS S3, Azure Blob, GCS

Lock mekanizması: DynamoDB (AWS)

Terraform Kullanım Alanları



- AWS altyapı kurulumu
- CI/CD entegrasyonu
- Kubernetes cluster kurulumu
- Multi-cloud altyapı yönetimi

Terraform ile wordpres auto scaling



Eğitim Değerlendirmesi

Sistem & Network Atölyesi – Kapanış

- VPC, ağ güvenliği, güvenli erişim, yük dengeleme, Temel Network-linux, **Auto Scaling** temelleri
- “Sürekli pratik” kültürü: küçük lab’lar kur → boz → nedenini anla
- Güncel trendler: **DevOps • Cloud • Containers • Observability**
- **Yapay zekâyı akıllı asistan gibi kullan** (anlayarak, kopyala-yapıştır değil)
- Bulut standart oldu: **AWS / Azure / Google Cloud**
- Temel **Linux + Shell/Python** her rolde fark yaratır
- IaC (Infrastructure as Code):
 - Terraform ile altyapıyı kodla, versiyonla, tekrarlanabilir ve denetlenebilir hâle getir



Ek Kaynaklar ve Öğrenmeye Devam

Pratik Yapabileceğin Siteler

1. OverTheWire - Bandit <https://overthewire.org/wargames/bandit/> → Linux komutları öğren (oyun formatında)

Host: bandit.labs.overthewire.org

Port: 2220

Username: bandit0

Password: bandit0

`ssh bandit0@bandit.labs.overthewire.org -p 2220`

2. Linux Journey <https://linuxjourney.com/> → Adım adım Linux eğitimi

3. Redhat <https://www.redhat.com/en/interactive-labs> → Red Hat interactive labs

4. KodeKloud <https://kodekloud.com/free-labs>

5. AWS Free Tier <https://aws.amazon.com/free/> → Kendi lab ortamını kur

Google Cloud Free (300\$ kredi / Free Tier): <https://cloud.google.com/free>

Microsoft Azure Free (200\$ + 12 ay ücretsiz): <https://azure.microsoft.com/free>



Bitirme Ödevi

✓ Ödev Seçenekleri

Sistem ve Network Atölyesinde Yaptığınız Projelerden Biri

- Daha önce birlikte yaptığımız projelerden herhangi birini seçip yeniden proje adımlarını yazarak.pdf dosyası olarak paylaşabilirsiniz.

Örnek

- VPC-Peering-Transit gateway kurulumu
- OpenVpn veya Aws Vpn Client kurulumu
- Aws Auto scaling örneği
- Custom VPC kurulumu
- ssh Tunnel Alistirmaları

Bitirme Ödevi



✓ Bitirme Ödevi Teslimi:

Bitirme ödevinizi tamamladıktan sonra, şu bilgileri içeren bir PDF dosyası oluşturmalsınız:

- Adınız Soyadınız
- Telefon Numaranız
- E-posta Adresiniz
- Projenizin GitHub Linki varsa
- Projenizin çalıştığını gösteren ekran görüntüleri

Hazırladığınız PDF dosyasını, size vereceğim Google Drive adresine yükleyerek teslim edebilirsiniz.